GSIP Technical Report

General Simulation Intelligence Platform System Architecture, Implementation, and Operations

Document version: 1.0 Classification: Technical Last updated: February 2025

Executive Summary

This technical report describes the General Simulation Intelligence Platform (GSIP), an

end-to-end c

Key technical principle: The Non-Negotiable Truth Architecture ensures that all numerical

results are p

---

1. Introduction and Problem Statement

1.1 Context

Decision-making in complex domains (finance, policy, operations) typically requires:

- Formal definition of objectives and constraints - Generation of many candidate

scenarios - E

Traditional tooling forces users to manually define objective functions, parameter

ranges, and

1.2 GSIP Approach

GSIP bridges the gap by:

1. Understanding intent Parsing natural language to extract optimization objectives,

constraints,

1.3 Non-Negotiable Truth Architecture

Rule: LLMs and AI agents may propose objectives, scenarios, and explanations; they must never fabrica

Enforcement:

- Only domain pack simulate() methods produce numerical outcomes. - Results are persisted to the databa

---

2. System Architecture

2.1 High-Level Diagram

```
Frontend Layer
```

Next.js Web

2.2 Component Summary

| Component | Technology | Responsibility | |-----------|------------|-----------------|

| Web App |

---

3. End-to-End Data Flow

3.1 Run Execution Flow

1. User submits a question in the chat (e.g. "Maximize portfolio returns while keeping risk low"). 2.

3.2 Evidence Flow

1. User uploads document Evidence service. 2. Text extracted (PDF/DOCX/text), chunked, embedded (

3.3 Key File References

| Purpose | File(s) | |---------|---------| | API entry, run creation | services/api/

---

4. Objective Formalization

4.1 Input and Output

- Input: User prompt (e.g. "Maximize my portfolio returns while keeping risk low"), optional dom

4.2 Implementation

- Domain detection: Keyword scoring against DOMAIN_KEYWORDS (finance-pack, spatial-pack, toy-pack); hi

Location: services/orchestrator/activities/formalizer.py, services/orc

---

5. Scenario Generation

5.1 Requirements

- Minimum 50 scenarios per run. - Diverse coverage: grid (e.g. 20%), Latin hypercube (e.g. 30%), r

5.2 Scenario Hash

Hash input: run_id, state, actions, seed, fidelity (and any other inputs to the simulation).

5.3 Persistence

Scenarios and scenario_instances persisted via orchestrator persistence activities; stored in Po

---

6. Domain Packs and Simulation Contract

6.1 Contract (compute/domain_packs/sdk/base.py)

Every domain pack implements:

```
class DomainPackBase:
```

```
def state_s
```

6.2 Fidelity Modes

- CHEAP: Fast approximation (e.g. 10 100 ms); initial exploration. - MID: Standard precision (e.

6.3 Implemented Packs

- ToyPack: 2D random walk; testing. - FinancePack: Portfolio backtest (e.g. SPY/BND/G

6.4 Registry

compute/domain_packs/sdk/registry.py creates pack instances by name/version. Packs registered vi

---

7. Sim Fabric (Execution Layer)

7.1 Role

- Ray-based distributed execution. - Worker pools load domain packs and run simulate(sta

7.2 Location

services/sim_fabric/executor.py SimulationWorker Ray actor; batch execution; result persistence.

---

8. Judge Service

8.1 Role

- Deterministic scoring: Rubric weights, threshold-based metric scoring, weighted aggregation.

8.2 Score Composition

Conceptually: final_score = raw_aggregate feasibility_multiplier (1 - total_penalty) -

constraint_p

---

9. Optimizer

9.1 Algorithms

- Bayesian optimization: Gaussian Process surrogate, Expected Improvement acquisition. -

NSGA-II: Ev

9.2 Integration

Orchestrator calls: initialize_optimizer, propose_next_batch, update_optimizer,

check_conv

---

10. Run Ledger (Truth Spine)

10.1 PostgreSQL Tables (Representative)

- runs: id, project_id, org_id, status, run_spec (JSONB), created_at, updated_at,

completed_a

10.2 Immutability and Provenance

- Run specification hashed and stored. - Scenario hashes and seeds recorded. - Artifact

checksums i

---

11. Security Model

- Authentication: JWT (e.g. HS256, configurable expiry). - Authorization: RBAC (e.g.

admin, analy

---

12. Observability

- Tracing: OpenTelemetry across API orchestrator sim fabric judge/evidence (as designed).

- Metrics: Pr

---

13. Quality Gates (Summary)

From docs/QUALITY_GATES.md:

- Reproducibility: Deterministic replay; hashed configs; seed policy and scenario seeds

recorded. - E

---

14. Configuration and Assumptions

From docs/assumptions.md (abbreviated):

- LLM: OpenAI-compatible API; MoE tiers (FAST/STANDARD/ADVANCED). - Embeddings: e.g.

all-MiniLM-L

---

15. Deployment Topology

- Containers (infra/docker-compose.yml): PostgreSQL (5433:5432), Redis (6379), MinIO

(9000, 9001

---

16. Testing

- Smoke e2e (tests/test_smoke_e2e.py): Different prompts different objectives; different

objectives di

---

17. Glossary

- Domain pack: Pluggable module defining state schema, action schema, simulation, and scoring for a

---

18. References

- Project docs: docs/architecture.md, docs/simulation-architecture-overview.md, docs/assum
HOW_IT_W

End of Technical Report